

เอกสารสำหรับใช้ในห้องประชุม — ผู้เข้าร่วมทุกคนควรอ่านภาค 1 มาก่อน

วาระประชุม

ตกลงสัญญา tRPC

19 เรื่องที่ต้องตกลง เรียงตามว่าอะไรโหล่กอะไร
เรื่องไหนต้องจบก่อนออกจากห้อง เรื่องไหนผลัดไปก่อนได้

เอกสารนี้ไม่ใช่บันทึกการประชุม แต่เป็น **สิ่งที่ต้องเตรียมก่อนประชุม**
แต่ละวาระมาพร้อมคำถามที่ต้องตอบ ตัวเลือกที่มี ข้อเสนอของฝ่าย
เทคนิค และผลที่จะตามมาถ้าไม่ตกลงในรอบนี้

จุดประสงค์คือให้ทุกคนเข้าห้องประชุมโดยรู้ตรงกันว่า กำลังจะตัดสินใจ
อะไร และ **ทำไมลำดับถึงเป็นแบบนี้** — ไม่ใช่มาถกกันว่าเรื่องไหนสำคัญ
กว่ากัน

เหตุผลเบื้องหลังของแต่ละวาระอยู่ในเอกสารคู่กัน `trpc-guide.tex`
ส่วนภาคผนวก ก ของเล่มนี้คือแบบฟอร์มบันทึกมติที่กรอกได้ระหว่าง
ประชุม

โครงการ: ระบบบริหารจัดการการยืม-คืนอุปกรณ์มหาวิทยาลัย (ULMs)

เวลาที่เสนอ: 90 นาที สำหรับภาค 1-2 (**P0** และ **P1**) · ภาค 3-4 ไม่ต้องคุยในรอบนี้

ปรับปรุงล่าสุด: 8 สิงหาคม 2569

สารบัญ

เอกสารนี้อ่านอย่างไร	3
1 PO ต้องจบก่อนออกจากห้อง	5
1 ว-01 · ไฟล์สัญญาจะเดินทางข้ามฝั่งอย่างไร	5
2 ว-02 · ใช้ zod เวอร์ชันเดียวกันทั้งสองฝั่ง	6
3 ว-03 · รูปร่างของ <code>ctx.user</code> และวิธีส่งข้อมูลยืนยันตัวตน	6
4 ว-04 · ใครเป็นเจ้าของสัญญา และการเปลี่ยนสัญญาทำอะไร	8
2 P1 ต้องจบภายในสัปดาห์นี้	9
5 ว-05 · โครงสร้างการตั้งชื่อ router กับ procedure	9
6 ว-06 · รูปแบบ error และรายการรหัสข้อผิดพลาดทางธุรกิจ	10
7 ว-07 · รูปแบบการแบ่งหน้าและการกรอง	11
8 ว-08 · รูปแบบของวันที่และเวลา	12
9 ว-09 · กฎการแปลงชื่อฟิลด์ และห้ามส่ง Prisma model ออกไปตรง ๆ	12
10 ว-10 · สถานะส่งออกเป็นสตริงคงที่ ไม่ใช่เลข key	13
3 P2 ตกลงตอนเริ่มทำ feature นั้น	14
11 ว-11 · ขั้นตอนการอัปเดตรูป	14
12 ว-12 · ความถี่ polling จริง และ <code>staleTime</code>	14
13 ว-13 · การรวมคำขอ เวลาหมดอายุ และการลองใหม่	14
14 ว-14 · ข้อความ error อยู่ฝั่งไหน	15
15 ว-15 · การบันทึกและตามรอยปัญหา	15

4	P3 ผลิตไปหลัง MVP ได้	16
16	ว-16 · การอัปเดตแบบเรียลไทม์	16
17	ว-17 · การจำกัดอัตราการเรียกและการป้องกันการยิงถี่	16
18	ว-18 · การทำเวอร์ชันของสัญญาอย่างเป็นทางการ	16
19	ว-19 · การทดสอบสัญญาอัตโนมัติ	16
5	วิธีดำเนินการประชุม	17
20	ตารางเวลาที่เสนอ	17
21	กติกาที่ทำให้ประชุมจบเป็นมติ	17
	ภาคผนวก	18
	ภาคผนวก	18
ก	แบบฟอร์มบันทึกมติ	18
ข	เช็กलिस्टก่อนเข้าประชุม	18
ค	โค้ดตัวอย่างของผลลัพธ์ — <code>docs/trpc-example/</code>	19
ง	สรุปทั้ง 19 วาระในหน้าเดียว	21

เอกสารนี้อ่านอย่างไร

เอกสารแบ่งเป็นห้าภาค เรียงตามว่าเรื่องไหนบล็อกเรื่องไหน ไม่ได้เรียงตาม ความยากหรือตามลำดับที่จะเขียนโค้ด เหตุผลคือในการประชุมที่มีเวลาจำกัด สิ่งที่ทำให้ ประชุมล้มเหลวไม่ใช่การตกลงไม่ได้ แต่คือ การใช้เวลาไปกับเรื่องที่รอได้ จนเรื่องที่รอไม่ได้ไม่ได้คุย

ระบบสีในเอกสารฉบับนี้ — สีระดับความเร่งด่วน

ระดับ	กำหนด	เกณฑ์ที่ใช้จัดระดับ
P0	ต้องจบก่อนออกจากห้อง	ถ้าไม่ตกลง ไม่มีใครเขียนโค้ดต่อได้เลย ทั้งสองฝั่ง
P1	ต้องจบภายในสัปดาห์นี้	เขียน procedure ตัวแรกได้ แต่ถ้ายังไม่ตกลง procedure ตัวที่ 2-30 จะออกมาไม่เหมือนกัน แล้วต้องกลับมาแก้ทั้งหมด
P2	ตกลงตอนเริ่มทำ feature นั้น	กระทบเฉพาะบางส่วนของระบบ ตัดสินใจช้าแล้วแก้ที่หลังได้ในต้นทุนที่ยอมรับได้
P3	ผลิตไปหลัง MVP ได้	ไม่กระทบการส่งงาน ถ้าคุยตอนนี้จะกินเวลาโดยไม่ได้อะไรกลับมา

ตารางที่ 1: สีระดับ — เกณฑ์คือ “ถ้าไม่ตกลงตอนนี้ จะต้องกลับมาแก้ของที่ทำไปแล้วมากแค่ไหน” ไม่ใช่ “เรื่องนี้สำคัญแค่ไหน”

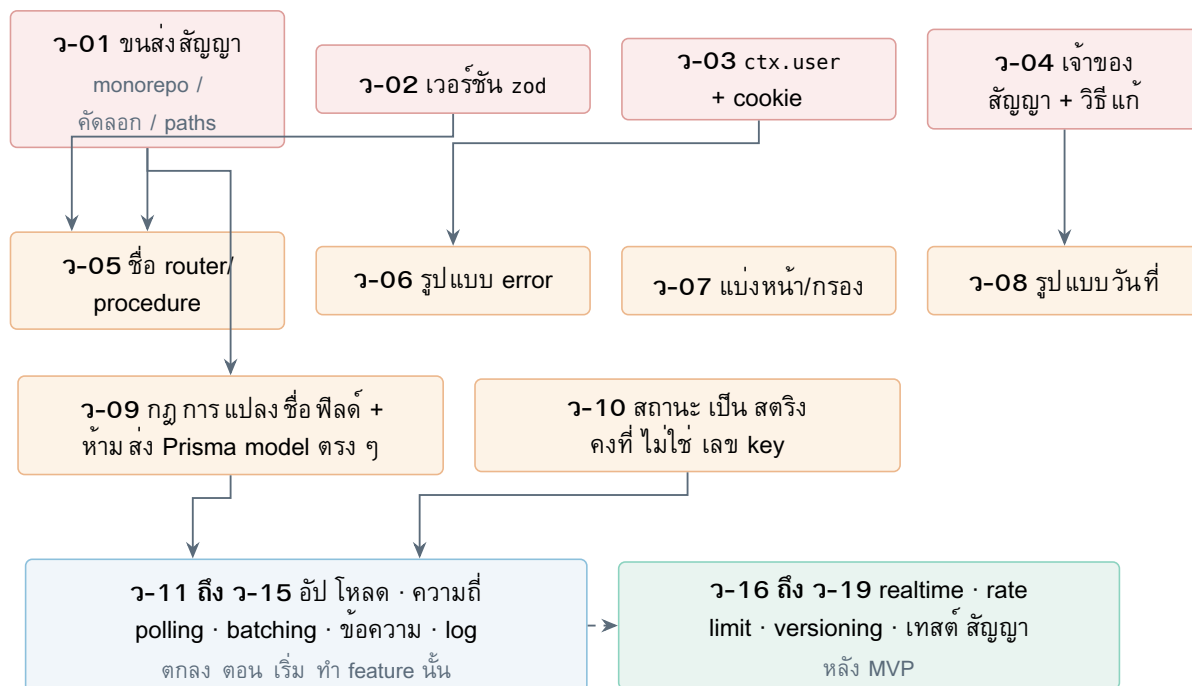
ทุกวาระสำคัญ — แต่ไม่ใช่ทุกวาระเร่งด่วน

วาระใน P3 ไม่ได้แปลว่าไม่สำคัญ การจำกัดอัตราการเรียก (rate limit) สำคัญมากในระบบจริง แต่มันแก้ที่หลังได้โดยไม่ต้องรื้อของเดิม ต่างจากการตั้งชื่อ procedure ที่ถ้าเปลี่ยนตอนมี 30 ตัวแล้วจะกระทบทุกไฟล์ทั้งสองฝั่ง

การจัดลำดับนี้ใช้เกณฑ์ ต้นทุนของการตัดสินใจช้า ไม่ใช่ความสำคัญ

แผนที่ความสัมพันธ์ — อะไรมีบล็อกอะไร

รูปข้างล่างคือเหตุผลของลำดับทั้งหมด ลูกศรอ่านว่า “ต้องตกลงข้อนี้ก่อน จึงจะตกลง ข้อถัดไปได้อย่างมีความหมาย”



รูปที่ 1: แผนที่วาระ — แฉวนสุดท้ายคือสิ่งที่ทุกอย่างข้างล่างพึ่งพา ถ้าประชุมได้แค่ 30 นาที ให้คุณเฉพาะแฉวนบน ลูกศรประหมายถึง “คุยได้แต่ไม่จำเป็น” คือภาค 3 กับ 4 ที่ไม่ควรอยู่ในวาระของรอบนี้

แต่ละวาระเขียนในรูปแบบเดียวกัน

เพื่อให้ประชุมเร็วและไม่หลงประเด็น ทุกวาระมีหัวข้อย่อยชุดเดียวกัน: คำถามที่ต้องตอบ → ทำไมอยู่ระดับนี้ → ตัวเลือก → ข้อเสนอ → ถ้าไม่ตกลงจะเกิดอะไร → ผลลัพธ์ที่ต้องจด

กล่องฟ้าปิดท้ายทุกวาระคือรายการที่ต้อง จดลงบันทึกจริง — ถ้าคุยจบแล้ว กรอกกล่องนั้นไม่ได้ แปลว่ายังไม่ได้ตกลง

ภาค 1

PO ต้องจบก่อนออกจากห้อง

สี่วาระนี้บล็อกทุกคน ถ้าไม่จบ ทั้ง frontend และ backend เขียนโค้ดที่จะได้ใช้จริงไม่ได้เลย เวลาที่เสนอ: 55 นาที

1 ว-01 · ไฟล์สัญญาจะเดินทางข้ามฝั่งอย่างไร

คำถามที่ต้องตอบ: frontend จะ import type ไฟล์สัญญาที่ backend สร้างขึ้นได้ด้วยวิธีไหน

ทำไมอยู่ระดับ **PO**: สัญญาของ tRPC คือ type ของ TypeScript ถ้าฝั่งหนึ่งมองไม่เห็นไฟล์ของอีกฝั่ง ก็ไม่มีสัญญา — มีแต่ REST ที่เขียนด้วยไวยากรณ์ของ tRPC ตอนนี้อยู่ไม่มี package.json สามไฟล์ที่ไม่รู้จักกัน และงานสองฝั่งอยู่คนละสาขา git

ตัวเลือก:

ทางเลือก	ต้องทำอะไร	ได้อะไร	เสียอะไร
ก. npm/pnpm workspaces	รวม frontend/, backend/, packages/contract/ เป็นรีโปเดียว ทุกคน install ใหม่	ถูกต้องที่สุด type ตรงกัน อัตโนมัติตลอด	ต้องรีโคโนโรจรีโป กระเทบทุกคนวันเดียวกัน จัดการสาขายากขึ้นช่วงแรก
ข. คัดลอกไฟล์ที่ถูกสร้าง	เขียนสคริปต์ sync:contract คัดลอกเข้า frontend/src/server-types/	เริ่มได้ทันที ไม่กระทบโครงสร้างเดิม	สัญญาเก่าได้เจียบ ๆ ถ้าลืมรันสคริปต์ ต้องมีการตรวจก่อน merge
ค. paths ใน tsconfig ชี้ข้ามโฟลเดอร์	ตั้งค่า tsconfig + vite.config	ไม่ต้องคัดลอก ไม่ต้อง publish	พังถ้าใครไม่ได้ clone ทั้งรีโป ตั้งค่าตอน vite build ยุ่ง

ตารางที่ 2: สามทางเลือกของการขนส่งสัญญา — ต่างกันที่จ่ายต้นทุนตอนนี้หรือจ่ายทีหลัง

ข้อเสนอ: เลือก ข. ก่อน โดยมีเงื่อนไขบังคับสองข้อ คือ (1) ต้องมีสคริปต์ npm run sync:contract ไม่ใช่คัดลอกด้วยมือ และ (2) ต้องมี tsc --noEmit ทั้งสองฝั่งเป็นเงื่อนไขก่อน merge แล้วค่อยพิจารณาย้ายไป ก. หลังส่ง MVP ถ้ายังมีเวลา

เหตุผลที่ไม่เลือก ก. ตั้งแต่แรกทั้งที่ถูกกว่าในระยะยาว: การรีโคโนโรจรีโป กลางเทอมทำให้ทุกคนหยุดงานพร้อมกันหนึ่งวัน และถ้าพลาดจะกินเวลามากกว่านั้น — เป็นความเสี่ยงที่ไม่คุ้มกับตารางเวลาที่เหลือ

ถ้าไม่ตกลงวันนี้: backend จะเขียน router ไปเรื่อย ๆ โดยที่ frontend ยังต้องเดา type เอง แล้วเมื่อถึงวันที่เชื่อมจริงจะพบว่าต้องแก้ ทั้งสองฝั่งพร้อมกัน — ซึ่งคือสถานการณ์ที่ tRPC ควรจะป้องกันตั้งแต่แรก

ผลลัพธ์ที่ต้องจดลงบันทึกก่อนปิดวาระนี้

- เลือกทางเลือก ก. / ข. / ค. —
- ตำแหน่งไฟล์สัญญาฝั่ง backend (ค่าของ autoSchemaFile) —
- ตำแหน่งปลายทางฝั่ง frontend —
- ใครเขียนสคริปต์ sync และเสร็จเมื่อไร —
- ตกลงหรือยังว่า tsc --noEmit เป็นเงื่อนไขก่อน merge

2 ว-02 · ใช้ zod เวอร์ชันเดียวกันทั้งสองฝั่ง

คำถามที่ต้องตอบ: จะยก frontend ขึ้น zod 4 หรือจะลด backend ลงมา 3 และใครทำ เสร็จเมื่อไร

ทำไมอยู่ระดับ PO: ตอนนี้ backend ใช้ zod 4.4 ส่วน frontend ใช้ 3.23 ไฟล์สัญญาที่ nestjs-trpc สร้างมีโค้ด zod อยู่ข้างใน เมื่อ frontend เอาไป compile ด้วยรุ่นใหญ่คนละรุ่นจะได้ error ที่อ่านไม่ออกกว่าสาเหตุคือเรื่องเวอร์ชัน — และจะเสียเวลาไล่หาสาเหตุกันทั้งทีม

ข้อเสนอ: ยก frontend ขึ้น zod 4 เพราะ backend เขียนโค้ดไปน้อยกว่ามาก และ zod 4 คือทิศทางที่ไลบรารีไปต่อ

งานนี้ไม่ใช่แค่เปลี่ยนตัวเลขใน package.json

ต้องตรวจของที่พึ่งพากันด้วย: frontend/src/schemas/loan-request.schema.ts ที่เขียนไว้แล้ว, แพ็กเกจ @hookform/resolvers ที่เชื่อม react-hook-form กับ zod และทุกจุดที่อ่านผลของ safeParse

ต้องทำเป็น PR แยกให้จบก่อน ไม่รวมกับงาน tRPC เพราะถ้ารวมกัน แล้วพึ่ง จะแยกไม่ออกว่าพึ่งเพราะเรื่องไหน

ถ้าไม่ตกลงวันนี้: คนแรกที่ยอมเชื่อมสองฝั่งจะติดกำแพงนี้ และจะเสียเวลา ครึ่งวันไปกับ error ที่ไม่ได้บอกสาเหตุจริง

ผลลัพธ์ที่ต้องจดลงบันทึกก่อนปิดวาระนี้

- เวอร์ชัน zod ที่ทั้งทีมใช้ —
- ใครทำการอัปเดต และ PR นี้จะ merge วันไหน —
- ยืนยันแล้วหรือยังว่า npm run typecheck ฝั่ง frontend ผ่านหลังอัปเดต

3 ว-03 · รูปร่างของ ctx.user และวิธีส่งข้อมูลยืนยันตัวตน

คำถามที่ต้องตอบ: object ที่ทุก procedure จะได้รับมีฟิลด์อะไรบ้าง และ token เดินทางมาถึง backend ได้อย่างไร

ทำไมอยู่ระดับ PO: ทุก procedure ที่ต้องล็อกอินพึ่ง ctx.user ถ้าเขียนไป 20 ตัวแล้วพบว่าขาดฟิลด์หนึ่ง

ต้องกลับไปแก้ทั้ง 20 ตัว พร้อมกับ service ที่เรียกใช้

ข้อเสนอเรื่องการส่ง token: เรื่องนี้ตัดสินใจไปแล้วโดยปริยาย — frontend/src/lib/api-client.ts ตั้ง credentials: "include" ไว้ทุก request แปลว่าใช้ httpOnly cookie สิ่งที่เหลือคือทำให้ฝั่ง backend รองรับ ซึ่งตอนนี้ main.ts ยังไม่มี enableCors เลย

ข้อเสนอเรื่องฟิลต์: เริ่มจากชุดนี้ และเพิ่มได้เฉพาะเมื่อมีเหตุผลชัดเจน

ฟิลต์	มาจาก	ใครใช้
accountKey	AccountInfo.AccountKey	ทุก query ที่กรองตามเจ้าของ
role	RoleInfo.RoleName	middleware ตรวจสอบสิทธิ์ทุกตัว
facultyKey	FacultyInfo	กฎการยืมที่ผูกกับสังกัด
creditScore	AccountInfo.UserCredit	ใช้หาระดับเครดิต (CreditTier) เพื่อคำนวณจำนวนวันยืมสูงสุด และจำนวนครั้งที่ต่ออายุได้

ตารางที่ 3: ชุดฟิลต์ขั้นต่ำที่เสนอ — ทุกฟิลต์ต้องตอบได้ว่ามีใครใช้ ไม่งั้นอย่าใส่

เครดิตต่ำ ไม่ได้ แปลว่ายืมไม่ได้ — แปลว่ายืมได้สั้นลง

ข้อนี้ต้องเข้าใจตรงกันทั้งทีมก่อนออกแบบ output เพราะมันเปลี่ยนว่า loan.create ควรตอบอะไรกลับมา

ใน schema.prisma ระดับ เครดิต เชื่อม กับ กฎ การ ยืม ผ่าน ตาราง BorrowConstraints ซึ่ง เก็บ MaxBorrowDate (จำนวนวันยืมสูงสุด) กับ MaxExtendTime (จำนวนครั้งที่ต่ออายุได้) ต่อคู่ “กฎการยืม × ระดับเครดิต” — **ไม่มีฟิลต์ไหนเลยที่บอกว่าเครดิตเท่าไรถึงห้ามยืม**

สิ่งที่ กั้น ไม่ ให้ ยืม คือ คนละ เรื่อง กัน: ตาราง Eligibility (สิทธิ์ ตาม สังกัด และ บทบาท) และ MinimumAuthorityLevel — ทั้งคู่ไม่เกี่ยวกับคะแนนเครดิต

creditScore ใน ctx เสี่ยงเป็นข้อมูลเก่า

ถ้า cron หักเครดิตรายวันระหว่างที่ผู้ใช้กำลังกรอกคำขอยืม ค่าที่ค้างใน ctx จะไม่ตรงกับฐานข้อมูล ผลคือหน้าจอบอกว่ายืมได้ 14 วัน แต่ระบบให้จริง 7 วัน

ต้องตกลงว่า procedure ที่ *คำนวณจำนวนวันยืมจริง* (เช่น loan.create) อ่านคะแนนสดจากฐานข้อมูลเสมอ ส่วนค่าใน ctx ใช้แสดงผลอย่างเดียว

ถ้าไม่ตกลงวันนี้: ไม่มีใครเขียน procedure ที่ต้องล็อกอินได้เลย ซึ่งคือเกือบทั้งหมดของระบบ

ผลลัพธ์ที่ต้องจดลงบันทึกก่อนปิดวาระนี้

- รายการฟิลต์ใน ctx.user (สรุปให้ครบ ห้ามค้าง “ไว้ค่อยเพิ่ม”) —
- ยืนยันใช้ httpOnly cookie —
- รายการ origin ที่จะอนุญาตใน CORS ทั้งตอนพัฒนาและตอนใช้จริง —
- อายุของ session และพฤติกรรมเมื่อหมดอายุ —
- ใครทำ context + CORS และเสร็จเมื่อไร

4 ว-04 · ใครเป็นเจ้าของสัญญา และการเปลี่ยนสัญญาอย่างไร

คำถามที่ต้องตอบ: เมื่อมีคนอยากเพิ่มฟิลด์หรือเปลี่ยนชื่อ procedure กระบวนการคืออะไร และใครมีสิทธิ์ตัดสินใจขั้นสุดท้าย

ทำไมอยู่ระดับ PO: นี่เป็นวาระเดียวในภาคนี้ที่ไม่ใช่เรื่องเทคนิค แต่เป็นวาระที่ทำให้อีกสามวาระมีผลจริง สัญญาที่ไม่มีเจ้าของจะถูกแก้ไขโดยใครก็ได้ โดยไม่มีใครรู้ และภายในสองสัปดาห์จะไม่มีใครเชื่อถือมันอีก

ข้อเสนอ: กำหนดสามอย่าง

1. **เจ้าของ** — หนึ่งคนต่อหนึ่งโดเมน (item, loan, approval, admin) เจ้าของเป็นคนอนุมัติการเปลี่ยน schema ของโดเมนนั้น ไม่ใช่คนเดียวคุมทั้งหมด เพราะจะกลายเป็นคอขวด
2. **การแบ่งชนิดการเปลี่ยน** — แยกให้ชัดว่าอันไหนแจ้งอย่างเดียวพอ อันไหนต้องคุยก่อน
3. **ช่องทางแจ้ง** — ที่เดียว ทุกคนเห็น ไม่ใช่แชตส่วนตัว

ชนิดการเปลี่ยน	ตัวอย่าง	ต้องทำอะไร
เพิ่มฟิลด์ที่ไม่บังคับ	เพิ่ม note? ใน output	แจ้งในช่องทางกลาง ไม่ต้องรออนุมัติ
เพิ่ม procedure ใหม่	เพิ่ม loan.extend	แจ้ง + เพิ่มแถวในตารางสัญญา
เปลี่ยนชื่อ / ลบฟิลด์ / เปลี่ยนชนิด	dueAt เป็น dueDate	ต้องคุยกับอีกฝั่งก่อน และแก้พร้อมกันใน PR เดียว
เปลี่ยนความหมายโดยรูปร่างเหมือนเดิม	status เดิมนับ “รอรับของ” เป็น pending เปลี่ยนเป็นสถานะใหม่	อันตรายที่สุด — compiler จับไม่ได้ ต้องคุยและต้องมีคนทดสอบ

ตารางที่ 4: สี่ชนิดการเปลี่ยน — แถวล่างสุดคือชนิดที่ระบบ type ช่วยอะไรไม่ได้เลย จึงต้องใช้กระบวนการของคนล้วน ๆ

ถ้าไม่ตกลงวันนี้: จะมีคนแก้ schema แล้วอีกฝั่งพังโดยไม่รู้ตัว เกิดขึ้นแน่นอน คำถามคือเกิดกี่ครั้งและเสียเวลาไปเท่าไร

ผลลัพธ์ที่ต้องจดลงบันทึกก่อนปิดวาระนี้

- เจ้าของแต่ละโดเมน —
- ช่องทางแจ้งการเปลี่ยนสัญญา —
- ยืนยันว่าการเปลี่ยนแบบ breaking ต้องแก้สองฝั่งใน PR เดียว —
- ใครดูแลตารางสัญญาให้ตรงกับโค้ด

ภาค 2

P1 ต้องจบภายในสัปดาห์นี้

หกวาระนี้ไม่บล็อกการเริ่มต้น แต่บล็อก “ความสม่ำเสมอ” — เขียน procedure ตัวแรกได้โดยไม่ต้องตกลง แต่ถ้าตกลงตอนมี 30 ตัวแล้วจะต้องแก้ทั้ง 30 เวลาที่เสนอ: 35 นาทีในรอบนี้ ที่เหลือแยกคุยเป็นรอบย่อย

5 ว-05 · โครงและการตั้งชื่อ router กับ procedure

คำถามที่ต้องตอบ: จัดกลุ่ม procedure ตามอะไร และใช้คำกริยาชุดไหน

ทำไมอยู่ระดับ **P1**: เปลี่ยนชื่อที่หลังต้องแก้ทุกจุดที่เรียกใช้ทั้งสองฝั่ง และเป็นการเปลี่ยนที่ไม่ได้ประโยชน์อะไรกลับมาจากความสม่ำเสมอ — งานที่ทีมมักไม่ยอมทำ แล้วปล่อยให้ชื่อมันต่อไปจนจบโครงการ

ข้อเสนอ 1 — จัดกลุ่มตามโดเมน ไม่ใช่ตามบทบาทผู้ใช้

แบบที่เสนอ (ตามโดเมน)	แบบที่ไม่เสนอ (ตามบทบาท)
approval.decide ตัวเดียว ตรวจสอบสิทธิ์ด้วย middleware	staff.approveLoan กับ supervisor.approveLoan สองตัว
ตรรกะการอนุมัติอยู่ที่เดียว แก่ครั้งเดียว	ตรรกะเดียวกันถูกคัดลอกสองที่ แก่ที่หนึ่งลืมอีกที่
เพิ่มบทบาทใหม่ = เพิ่ม middleware	เพิ่มบทบาทใหม่ = เพิ่ม router ทั้งชุด

ตารางที่ 5: จัดกลุ่มตามโดเมนชนะเพราะบทบาทเปลี่ยนบ่อยกว่าโดเมน — ในระบบนี้มี 4-5 บทบาทที่ยังอาจปรับได้อีก

โดเมนที่เสนอ: auth, item, loan, reservation, approval, appeal, credit, inspection, notification, report, admin

ข้อเสนอ 2 — ชุดคำกริยาที่ใช้ซ้ำทั้งระบบ

list · getById · create · update · cancel · decide · confirm — ห้ามปน getAll กับ fetchList กับ findMany ในความหมายเดียวกัน

ผลลัพธ์ที่ต้องจดลงบันทึกก่อนปิดวาระนี้

- รายชื่อโดเมน (router) ที่ตกลง —
- ชุดคำกริยามาตรฐาน —
- ยืนยันว่าตรวจสอบสิทธิ์ด้วย middleware ไม่ใช่แยก router ตามบทบาท

6 ว-06 · รูปแบบ error และรายการรหัสข้อผิดพลาดทางธุรกิจ

คำถามที่ต้องตอบ: เมื่อ “ของถูกคนอื่นจองตัดหน้าไปแล้ว” backend ส่งอะไรกลับมา และ frontend แปลงเป็นข้อความที่ไหน

ทำไมอยู่ระดับ P1: frontend ต้องเขียน UI รองรับแต่ละกรณี รายการรหัสจึงเป็นส่วนหนึ่งของ output ไม่ใช่รายละเอียดปลีกย่อย — ถ้าไม่มีรายการตั้งแต่ต้น frontend จะเขียนได้แค่ “เกิดข้อผิดพลาด” แล้วต้องกลับมาแก้ที่หลังทุกหน้า

ข้อเสนอ: สอง ชั้น — ชั้นนอกใช้รหัสมาตรฐานของ tRPC (UNAUTHORIZED, FORBIDDEN, NOT_FOUND, CONFLICT, BAD_REQUEST) ชั้นในใส่รหัสธุรกิจของเราเอง

รหัสธุรกิจ	รหัส tRPC	frontend ต้องทำอะไร
NOT_ELIGIBLE	FORBIDDEN	บอกว่าสังกัด/บทบาทนี้ยืมของชั้นนี้ไม่ได้
ITEM_UNAVAILABLE	CONFLICT	แสดงวันที่พร้อมให้ยืมถัดไป
SLOT_TAKEN	CONFLICT	refetch สล็อตแล้วให้เลือกใหม่
LOAN_PERIOD_EXCEEDS_LIMIT	BAD_REQUEST	บอกเพดานวันยืมของระดับเครดิตปัจจุบัน แล้วให้เลือกวันใหม่
EXTENSION_QUOTA_EXCEEDED	FORBIDDEN	บอกจำนวนครั้งที่ใช้ไปแล้วจากเพดานของระดับเครดิต
ALREADY_DECIDED	CONFLICT	refetch คิว — มีคนตัดสินใจไปแล้ว
APPEAL_WINDOW_CLOSED	FORBIDDEN	บอกวันที่หมดเขต

ตารางที่ 6: ร่างรายการรหัส — ยังไม่ครบ ต้องเติมให้ครบตอนแปลงเอกสารรายการเรียกใช้งานเป็นตารางสัญญา สังเกตว่า ไม่มี รหัสว่า “เครดิตต่ำจนยืมไม่ได้” เพราะกฎแบบนั้นไม่มีในระบบ

คำถามที่ยังไม่มีใครตอบ: เครดิตต่ำแล้วขอยืมนานเกินเพดาน จะ “ปฏิเสธ” หรือ “ให้เท่าที่ได้”

เพราะเครดิตกำหนดแค่ *ความยาว* ของการยืม ไม่ใช่สิทธิ์ยืม จึงเกิดทางแยกที่ กระทบทั้ง error และ output ของ `loan.create`

- ก. ปฏิเสธ — โยน `LOAN_PERIOD_EXCEEDS_LIMIT` พร้อมเพดาน แล้วให้ผู้ใช้เลือกวันใหม่ · ชัดเจน ไม่มีเซอร์ไพรส์ แต่เพิ่มขั้นตอนให้ผู้ใช้
- ข. ตัดให้อัตโนมัติ — อนุมัติแต่ย่นวันคืนให้เท่าเพดาน · สั้นไหลกว่า แต่ output ต้องบอกกลับมาให้ชัดว่าได้วันจริง และถูกย่นเพราะอะไร ไม่งั้นผู้ใช้จะคืนของสาย

ข้อเสนอ: เลือก ก. สำหรับ MVP เพราะแสดงผลง่ายกว่าและไม่ต้องออกแบบ ช่องทางแจ้งเตือนเพิ่ม — แต่ไม่ว่าจะเลือกทางไหน output ของ `loan.create` ต้องมีวันครบกำหนดจริงเสมอ ห้ามให้ frontend คำนวณเอง

ห้าม backend ส่งข้อความภาษาไทยมาให้แสดงตรง ๆ

โปรเจกต์ลง `i18next` และ `react-i18next` ไว้แล้ว แปลว่าตั้งใจรองรับ หลายภาษา ถ้า backend ส่ง “ขอยืมได้ไม่เกิน 7 วัน” มาแล้ว frontend เอาไปแสดงเลย ระบบหลายภาษาจะใช้ไม่ได้ทันที และข้อความ

จะกระจายอยู่ในโค้ดสองที่

backend ส่ง รหัส กับ ข้อมูลประกอบ (เช่น เพดานวันยืมของ ระดับเครดิตปัจจุบัน จำนวนวันที่ขอมมา) แล้วให้ frontend ประกอบเป็นประโยค

ผลลัพธ์ที่ต้องจดลงบันทึกก่อนปิดวาระนี้

- ยืนยันโครงสร้าง error สองชั้น —
- รายการรหัสธุรกิจฉบับเริ่มต้น และใครเป็นคนเติมให้ครบ —
- ยืนยันว่าข้อความภาษาไทยอยู่ฝั่ง frontend เท่านั้น —
- เลือก ก. ปฏิเสธ หรือ ข. ตัดวันยืมให้อัตโนมัติ เมื่อขอเกินเพดานของระดับเครดิต —
- รูปแบบข้อมูลประกอบที่แนบไปกับ error

7 ว-07 · รูปแบบการแบ่งหน้าและการกรอง

คำถามที่ต้องตอบ: procedure ที่คืนรายการ จะรับพารามิเตอร์ชุดไหน และคืนอะไรกลับมา

ทำไมอยู่ระดับ **P1**: เอกสารรายการเรียกใช้งานมีรายการที่ต้องแบ่งหน้า อย่างน้อย 5 จุด ถ้าแต่ละคนออกแบบเอง frontend จะต้องเขียน component ตารางคนละแบบสำหรับแต่ละหน้า

แบบ	หน้าตา	เหมาะกับ
ตามเลขหน้า	รับ page, pageSize, sort, q คืน items, total, page, pageSize	หน้าจอฝั่ง staff/admin ที่ต้องเห็นว่ามีทั้งหมดกี่หน้า และกระโดดไปหน้าที่ต้องการได้
ตามตัวชี้ (cursor)	รับ cursor, limit คืน items, nextCursor	เลื่อนลงเรื่อย ๆ บนมือถือ ประสิทธิภาพดีกว่าเมื่อข้อมูลเยอะมาก

ตารางที่ 7: สองแบบ — ข้อเสนอคือแบบเลขหน้า เพราะหน้าจอส่วนใหญ่ในระบบนี้เป็นตารางงานของเจ้าหน้าที่

ข้อเสนอ: ใช้แบบเลขหน้าทั้งระบบ กำหนด pageSize ปริยาย 20 และเพดาน 100 (กันคนยิงขอทีเดียวหมื่นแถว)

ผลลัพธ์ที่ต้องจดลงบันทึกก่อนปิดวาระนี้

- แบบที่เลือก และชื่อฟิลด์ที่ตกลง —
- ค่าปริยายและเพดานของ pageSize —
- รูปแบบการส่งเงื่อนไขกรอง (แยกเป็นฟิลด์ หรือรวมเป็น object filters)

8 ว-08 · รูปแบบของวันที่และเวลา

คำถามที่ต้องตอบ: ส่ง Date จริงผ่าน superjson หรือส่งเป็น สตริง ISO

ทำไมอยู่ระดับ **P1**: schema.prisma มีฟิลด์ DateTime กระจายอยู่ 9 ตารางหลัก และระบบนี้ “เวลา” คือ แก่นของตรรกะธุรกิจ — กำหนดคืน การเลยกำหนด การหักเครดิตรายวัน อายุบทลงโทษ ถ้าตกลงไม่ตรงกัน บั๊กจะไปโผล่ที่ การคำนวณค่าปรับ

ข้อเสนอ: สตริง ISO เพราะดีบักง่าย (เปิด Network tab แล้วอ่านออก ทันที) ไม่ต้องตั้งค่าให้ตรงกันสองฝั่ง และทีมมี date-fns อยู่แล้ว

สิ่งที่สำคัญกว่าการเลือกแบบไหน คือห้ามมีข้อยกเว้น

procedure ที่ส่ง Date จริงปนกับ procedure ที่ส่งสตริง คือ สถานการณ์ที่แย่ที่สุดในสามแบบ เพราะ frontend จะไม่รู้ว่าจะต้อง parseISO เมื่อไร — และ type จะบอกว่าถูกทั้งสองแบบ

นอกจากรูปแบบแล้วต้องตกลงอีกสองเรื่องที่สัมพันธ์กันบ่อย: **เขตเวลา** (เก็บเป็น UTC แล้วแปลงตอนแสดงผลใช้ใหม่) และ **ค่าที่เป็นวันล้น** เช่นวันครบกำหนดคืน — เป็นวันที่อย่างเดียวยังหรือมีเวลาด้วย และถ้ามีเวลา กำหนดคืน “วันที่ 20” หมายถึงกี่โมง

ผลลัพธ์ที่ต้องตกลงบันทึกก่อนปิดวาระนี้

- รูปแบบที่เลือก —
- เขตเวลาที่เก็บและที่แสดง —
- ฟิลด์ที่เป็นวันล้น จัดการอย่างไร —
- เวลาตัดของ “ครบกำหนดคืน”

9 ว-09 · กฎการแปลงชื่อฟิลด์ และห้ามส่ง Prisma model ออกไปตรง ๆ

คำถามที่ต้องตอบ: output จะพูดภาษารฐานข้อมูลหรือภาษา frontend

ทำไมอยู่ระดับ **P1**: schema.prisma ตั้งชื่อแบบ PascalCase (UserFName, AccountKey) ส่วน frontend/src/types/domain.ts ใช้ camelCase (firstName, id) ถ้าไม่ตกลงตอนนี้ procedure ที่เขียนก่อนจะส่งชื่อแบบหนึ่ง ตัวที่เขียนทีหลังส่งอีกแบบ

ข้อเสนอ: output เป็นภาษา frontend เสมอ (camelCase) และห้าม return Prisma model ตรง ๆ ในทุกกรณี

เหตุผลที่ห้ามไม่ใช่แค่เรื่องความสวยงามของชื่อ

ตาราง AccountInfo มีฟิลด์ HashedPassword อยู่ในตารางเดียวกับชื่อ และอีเมล การ findUnique แล้วส่งกลับตรง ๆ คือการส่ง hash รหัสผ่าน ออกไปที่เบราว์เซอร์ — และไม่มีอะไรในระบบเตือนเลย ประโยชน์อีกข้อคือฐานข้อมูลเปลี่ยนได้โดยหน้าเว็บไม่พัง ซึ่งสำคัญกับรีปนี้เป็นพิเศษ เพราะ schema ยัง

อยู่ในช่วงที่อาจปรับอีก

ผลลัพธ์ที่ต้องจดลงบันทึกก่อนปิดวาระนี้

- ยืนยันว่า output ใช้ camelCase —
- ยืนยันกฎ “ห้ามส่ง Prisma model ตรง ๆ” และใครตรวจตอนรีวิว —
- ที่เก็บ output schema กลางที่นำไป .pick()/.extend() ต่อได้

10 ว-10 · สถานะส่งออกเป็นสตริงคงที่ ไม่ใช่เลข key

คำถามที่ต้องตอบ: frontend จะได้รับสถานะเป็นอะไร

ทำไม อยู่ระดับ **P1**: schema.prisma ไม่มี enum เลย สัก ตัว — สถานะ ทุก ชนิด เป็น ตาราง แยก (ApproveStatus, ResourceStatus, ConditionType, NotificationType, PenaltyType และ แม้แต่ RoleInfo) ขณะที่ frontend นิยามไว้ เป็นสตริง เช่น "available" | "reserved" | "borrowed"

ส่งเลข primary key ของสถานะออกไป = บั๊กที่จะโผล่ตอนย้ายเครื่อง

ApproveStatusKey = 2 หมายถึง “อนุมัติแล้ว” บนฐานข้อมูลเครื่องหนึ่ง แต่ถ้าอีกเครื่อง seed คนละลำดับ เลข 2 อาจกลายเป็น “ปฏิเสธ”
โค้ด if (status === 2) จะคอมไพล์ผ่านทุกครั้งและผิดเจียบ ๆ — เป็นบั๊กที่จะเจอตอนสาธิตงาน ไม่ใช่ตอนพัฒนา

ข้อเสนอ: output ใช้ z.enum([...]) ที่เป็นสตริงคงที่เสมอ และต้องตกลง รายการค่าที่เป็นไปได้ของแต่ละสถานะ ให้ครบในวาระนี้ เพราะ frontend ต้องเขียน UI ครอบคลุมทุกค่า

ผลลัพธ์ที่ต้องจดลงบันทึกก่อนปิดวาระนี้

- ยืนยันว่าสถานะส่งเป็นสตริง —
- รายการค่าของ ApproveStatus —
- รายการค่าของ ResourceStatus —
- รายการค่าของ RoleInfo —
- ใครดูแลให้ตารางในฐานข้อมูลกับ z.enum ไม่หลุดจากกัน

ภาค 3

P2 ตกลงตอนเริ่มทำ feature นั้น

ห้าวาระนี้ *ไม่ต้องคุยในการประชุมรอบนี้* เอกสารระบุไว้เพื่อให้ทุกคน เห็นว่ามันไม่ได้ถูกลิ้ม และรู้ว่าจะถูกหยิบมาคุยเมื่อไร — การเขียนสิ่งที่ยัง ไม่ตัดสินใจลงกระดาษ คือสิ่งที่ทำให้ “ผลัดไปก่อน” ต่างจาก “ลิ้ม”

11 ว-11 · ขั้นตอนการอัปโหลดรูป

คำถาม: รูปตอนรับของและตอนคืนของเดินทางอย่างไร เก็บที่ไหน

ทำไมผลัดได้: tRPC ส่งไฟล์ไม่ได้อยู่แล้ว จึงต้องใช้เส้นทาง REST แยก ซึ่ง `api-client.ts` มี `uploadFile` รองรับไว้แล้ว — โครงสร้างส่วนนี้ไม่ขึ้นกับการตัดสินใจเรื่องสัญญา

คุยเมื่อ: เริ่มทำ feature รับ-คืนของ

ประเด็นที่ต้องเตรียมไว้: ที่เก็บไฟล์ · การขอ URL ชั่วคราว · เพดานขนาดและชนิดไฟล์ · การจัดการไฟล์ที่อัปแล้วแต่ไม่มีแถวใน Images ซีถึง (ผู้ใช้ปิดเบราว์เซอร์กลางคัน) · การบีบอัดรูปก่อนอัปจากมือถือ

12 ว-12 · ความถี่ polling จริง และ staleTime

คำถาม: ตัวเลขในเอกสารรายการเรียกใช้งานใช้ได้จริงไหม

ทำไมผลัดได้: เปลี่ยนตัวเลขเป็นการแก้บรรทัดเดียวฝั่ง frontend ไม่กระทบสัญญา

คุยเมื่อ: หลังต่อหน้าจอที่ polling หน้าแรกได้แล้ว และวัดภาระจริงได้

ประเด็น: 10–15 วินาทีสำหรับของคงเหลือเหมาะกับจำนวนผู้ใช้จริงหรือไม่ · staleTime ยาวสำหรับข้อมูลหลัก (หมวดหมู่ tier ระดับความเสียหาย) · หยุด polling เมื่อผู้ใช้สลับแท็บออกไป

13 ว-13 · การรวมคำขอ เวลาหมดอายุ และการลงใหม่

คำถาม: ใช้ `httpBatchLink` ทั้งหมด หรือแยกเส้นสำหรับ query ที่ถี่

ทำไมผลัดได้: เปลี่ยน link เป็นการแก้ไฟล์ตั้งค่าไฟล์เดียว

คุยเมื่อ: พบว่าหน้าใดหน้าหนึ่งซ้ำผิดปกติ

ประเด็น: query ที่ poll ทุก 10 วินาทีไปรวมก่อนกับรายงานสถิติ จะถูกลากให้ซ้ำตาม · จะลงใหม่กี่ครั้งเมื่อเครือข่ายหลุด · mutation ต้องไม่ลงใหม่อัตโนมัติ (ยืมซ้ำสองครั้ง)

14 ว-14 · ข้อความ error อยู่ฝั่งไหน

คำถาม: แผนผังการไหลธุรกิจไปเป็นประโยชน์เก็บที่ไหน

ทำไมผลัดได้: เมื่อ ว-06 ตกลงแล้วว่าส่งเป็นรหัส ที่เหลือเป็นงานภายในของ frontend ล้วน ๆ

คุยเมื่อ: เริ่มทำหน้าจอก็มี error หลายกรณี

ประเด็น: รวมเข้ากับ lib/error-messages.ts ที่มีอยู่แล้ว หรือย้าย เข้าไฟล์แปลของ i18next · ข้อความปริยายเมื่อเจอรหัสที่ยังไม่ได้แปล

15 ว-15 · การบันทึกและตามรอยปัญหา

คำถาม: เมื่อผู้ใช้แจ้งว่า “กดปุ่มแล้วขึ้น error” จะหาสาเหตุอย่างไร

ทำไมผลัดได้: เพิ่มทีหลังได้โดยไม่กระทบสัญญา

คุยเมื่อ: ใกล้ส่งงาน หรือเมื่อเริ่มมีผู้ใช้จริงทดลอง

ประเด็น: log ชื่อ procedure เวลาที่ใช้ และรหัสข้อผิดพลาด · รหัสอ้างอิงต่อ request ที่แสดงให้ผู้ใช้บอกกลับมาได้ · ระวังอย่า log ข้อมูลส่วนบุคคล

ภาค 4

P3 ผลัดไปหลัง MVP ได้

สัปดาห์นี้สำคัญในระบบที่ใช้งานจริง แต่ไม่กระทบการส่งงาน และที่สำคัญคือ *แก้ที่หลังได้โดยไม่ต้องรื้อของเดิม* — เขียนไว้เพื่อไม่ให้หายไปจากความทรงจำของทีม

16 ว-16 · การอัปเดตแบบเรียลไทม์

ทีมเลือก polling ไปแล้วในเอกสารรายการเรียกใช้งาน ซึ่งเป็นการตัดสินใจที่ เหมาะกับขนาดของระบบและเวลาที่มี tRPC รองรับ subscription ได้อ่อนนาคด ต้องการ และการย้ายจาก polling ไป subscription ไม่กระทบรูปร่างของ output ที่ตกลงกันไว้ — นี่คือเหตุผลที่ผลัดได้อย่างปลอดภัย

17 ว-17 · การจำกัดอัตราการเรียกและการป้องกันการยิงถี่

query ที่ poll ทุก 10 วินาทีคุณจำนวนผู้ใช้พร้อมกันเป็นภาระที่คาดเดาได้ ส่วนที่คาดเดาไม่ได้คือคนที่ยิงตรงเข้ามา เพิ่มการจำกัดอัตราที่หลังได้ที่ชั้น middleware โดยไม่แตะสัญญา

18 ว-18 · การทำเวอร์ชันของสัญญาอย่างเป็นทางการ

ตราบไคที่ยังส่งพร้อมกันทั้งสองฝั่ง ไม่ต้องมีเวอร์ชัน เรื่องนี้จำเป็นเมื่อมี client ที่อัปเดตไม่พร้อมกัน เช่นแอปมือถือที่ผู้ใช้ยังไม่กดอัปเดต — ยังไม่ใช่สถานการณ์ของโครงการนี้

19 ว-19 · การทดสอบสัญญาอัตโนมัติ

tsc --noEmit ที่ตกลงใน ว-01 ครอบคลุมความเสี่ยงหลักไปแล้ว ส่วนการเขียน เทสต์ที่ยิง procedure จริงแล้วตรวจ output กับ schema เป็นขั้นถัดไป ที่เพิ่มได้เมื่อมีเวลา — backend มี jest กับ supertest ติดตั้งไว้แล้ว

ภาค 5

วิธีดำเนินการประชุม

คำถามที่ภาคนี้ตอบ: ทำอย่างไรให้ 90 นาทีนี้จบด้วยมติ ไม่ใช่จบด้วย “เดียวไปคุยกันต่อ”

20 ตารางเวลาที่เสนอ

เวลา	วาระ	หมายเหตุ
5 นาที	ทบทวนแผนที่ในรูปที่ 1	ให้ทุกคนเห็นตรงกันว่าคุยอะไรบ้าง
20 นาที	PO ว-01 ขนส่งสัญญา	วาระที่ยาวที่สุด เพราะมีทางเลือกจริง
5 นาที	PO ว-02 เวอร์ชัน zod	น่าจะจบเร็ว มีข้อเสนอชัดอยู่แล้ว
20 นาที	PO ว-03 ctx.user + cookie	ต้องได้รายการฟิลต์ที่ครบ
10 นาที	PO ว-04 เจ้าของสัญญา	เรื่องคน ไม่ใช่เรื่องเทคนิค
25 นาที	P1 ว-05 ถึง ว-10 รวบคุย	ถ้าเวลาไม่พอ ให้ยกไปร่อย่อย
5 นาที	ทวนมติ + ผู้รับผิดชอบ + กำหนดเสร็จ	ห้ามข้าม

ตารางที่ 8: 90 นาที — สังเกตว่า **PO** กินเวลา 55 นาที จากทั้งหมด 90 นาที ซึ่งเป็นสัดส่วนที่ตั้งใจ

21 กติกาที่ทำให้ประชุมจบเป็นมติ

1. ทุกวาระต้องได้ครบสามอย่าง — ข้อตกลง · ผู้รับผิดชอบ · วันเสร็จ ขาดข้อใดข้อหนึ่งแปลว่ายังไม่จบวาระ
2. มีคนจับเวลา และมีสิทธิ์ตัดบทได้จริง
3. ถ้าเถียงกันเกิน 5 นาทีโดยไม่มีข้อมูลใหม่ ให้เลือกข้อเสนอในเอกสาร ไปก่อน แล้วบันทึกว่าเป็นการตัดสินใจแบบทบทวนได้ — การตัดสินใจที่กลับได้ ไม่ควรใช้เวลาเท่าการตัดสินใจที่กลับไม่ได้
4. เรื่องที่ไม่อยู่ในภาค 1-2 ห้ามคุย จดลงรายการค้างแล้วไปต่อ
5. ผู้เข้าร่วมต้องอ่านภาค 1 มาก่อน — ถ้ายังไม่มีใครอ่าน ให้เลื่อนประมุติกว่าใช้เวลาครึ่งหนึ่งไปกับการอธิบาย

สิ่งที่วัดว่าประชุมนี้สำเร็จ

กรอกกล่องฟ้า “ผลลัพธ์ที่ต้องจดลงบันทึก” ของวาระ ว-01 ถึง ว-04 ได้ครบทุกช่อง — ไม่ใช่ “คุยครบทุกวาระ”

ภาคผนวก

ก แบบฟอร์มบันทึกมติ

พิมพ์หน้านี้ออกมาหรือเปิดดูไว้ระหว่างประชุม กรอกให้ครบทุกช่องก่อนปิดวาระ

วาระ	ข้อตกลง	รายละเอียด/เงื่อนไข	ผู้รับผิดชอบ	วันเสร็จ
ว-01				
ว-02				
ว-03				
ว-04				
ว-05				
ว-06				
ว-07				
ว-08				
ว-09				
ว-10				

ข เช็คลิสต์ก่อนเข้าประชุม

#	สิ่งที่ต้องเตรียม	ใคร
1	อ่านภาค 1 ของเอกสารนี้ (PO สีวาระ)	ทุกคน
2	อ่านภาค 4 ของ trpc-guide.tex (ความเสี่ยง 8 ข้อ)	อย่างน้อยฝ่ายเทคนิค
3	เปิดดู frontend/package.json กับ backend-preview/backend/package.json เทียบเวอร์ชัน zod จริง	คนเสนอ ว-02
4	เตรียมรายการฟิลด์ที่คิดว่า ctx.user ต้องมี	backend
5	เตรียมรายการหน้าจที่ต้องใช้ข้อมูลผู้ใช้	frontend
6	เปิด schema.prisma ค้างไว้ที่ตารางสถานะ (ApproveStatus, ResourceStatus)	คนนำ ว-10
7	เปิด docs/trpc-example/ ค้างไว้ — ตัวอย่างโค้ดของทุกวาระ (ภาคผนวก ค)	ทุกคน
8	เตรียมกระดาษ/ไฟล์สำหรับกรอกแบบฟอร์มภาคผนวก ก	ผู้จัดบันทึก

ตารางที่ 10: เช็คลิสต์ — ข้อ 1 เป็นข้อที่ตัดสินว่าประชุมจะได้ผลหรือไม่

ค โค้ดตัวอย่างของผลลัพธ์ — docs/trpc-example/

มีไฟล์เตอร์ docs/trpc-example/ ที่เขียน หน้าตาของโค้ดเมื่อประชุม จบแล้ว ไว้ให้ครบทุกวาระใน **PO** และ **P1** — เป็นไฟล์จริงที่ก๊อปปี้ไปใช้ได้ ไม่ใช่ pseudocode

จุดประสงค์คือให้ดูปลายทางก่อนถกเถียง เพราะการเถียงว่า “แบ่งหน้าแบบไหนดี” บนกระดานกินเวลากว่าการดูโค้ดสองแบบวางข้างกันมาก

วาระ	ไฟล์ที่แสดงผลลัพธ์	ถ้าที่ประชุมเลือกอย่างอื่น
ว-01	backend/src/app.module.ts scripts/sync-contract.sh frontend/src/server-types/	เลือก workspaces → ลบสคริปต์ ย้ายสัญญาไป packages/contract/
ว-02	ส่วน “ก่อนใช้งาน” ใน README.md	เลือกลด backend เป็น zod 3 → แก่ z.email() กับ z.iso.datetime()
ว-03	backend/src/trpc/context.ts backend/src/trpc/auth.middleware.ts backend/src/main.ts	แก้ฟิลด์ใน TrpcUser ที่เดียว แล้ว compiler จะฟ้องทุกจุดที่ต้องตามแก้
ว-04	process/CONTRACT-OWNERS.md process/pull_request_template.md	กรอกชื่อเจ้าของในตารางระหว่างประชุม
ว-05	backend/src/loan/loan.router.ts backend/src/auth/auth.router.ts	เปลี่ยนชุดคำกริยา → แก่ชื่อเมทอดในทุก router
ว-06	common/errors/error-codes.ts common/errors/business-error.ts frontend/src/lib/error-messages.ts	เลือก “ตัดวันยืมอัตโนมัติ” แทนปฏิเสธ → แก่บล็อกใน loan.service.ts ตามคอมเมนต์ที่เขียนไว้ให้แล้ว
ว-07	common/schemas/pagination.schema.ts	เลือก cursor → เปลี่ยน paginated() เป็นคืน nextCursor
ว-08	common/schemas/datetime.schema.ts	เลือก superjson → ลบไฟล์นี้ แล้วตั้ง transformer สองฝั่ง
ว-09	common/schemas/user.schema.ts common/mappers/user.mapper.ts	— (แทบไม่มีทางเลือกอื่นที่ปลอดภัย)
ว-10	common/schemas/status.schema.ts	เติมรายการค่าให้ตรงกับตารางสถานะจริงในฐานข้อมูล
รวม	CONTRACT.md ตารางสัญญาที่คนอ่านได้ frontend/.../use-loans.ts ตัวอย่างการใช้จริง	อัปเดตทุกครั้งที่เพิ่ม procedure

ตารางที่ 11: แผนที่จากวาระไปหาไฟล์ — ทุกไฟล์มีคอมเมนต์หัวไฟล์บอกว่าตอบวาระไหน สมมติว่าเลือกอะไร และปลายทางจริงอยู่ไหน

โค้ดชุดนั้นเขียนบน ข้อเสนอ ยังไม่ใช้มิติ

ทุกไฟล์เขียนบนสมมติฐานว่าที่ประชุมเลือกตามข้อเสนอในเอกสารฉบับนี้ ถ้าที่ประชุม ตัดสินต่างออกไป คอลัมน์ขวาสุดของตารางบอกไว้แล้วว่าต้องแก้ไฟล์ไหน

สถานะการตรวจสอบ: ไฟล์ .ts ทั้ง 15 ไฟล์ผ่าน `tsc --strict` โดยไม่มี syntax error เหลือ แต่ ยังไม่ได้พิสูจน์ว่า type ตรงกันจริง เพราะเครื่องที่สร้างยังไม่ได้ลง `node_modules` ของทั้งสองโปรเจกต์ — ต้องรัน `tsc --noEmit` ซ้ำหลังคัดลอกไปวางในโปรเจกต์จริง

ง สรุปทั้ง 19 วาระในหน้าเดียว

วาระ	ระดับ	เรื่อง	ตกลงเมื่อไร
ว-01	P0	ไฟล์สัญญาเดินทางข้ามฝั่งอย่างไร	ก่อนออกจากห้อง
ว-02	P0	เวอร์ชัน zod ตรงกันทั้งสองฝั่ง	ก่อนออกจากห้อง
ว-03	P0	รูปร่าง ctx.user + cookie/CORS	ก่อนออกจากห้อง
ว-04	P0	เจ้าของสัญญาและวิธีขอเปลี่ยน	ก่อนออกจากห้อง
ว-05	P1	ชื่อ router และ procedure	ในสัปดาห์นี้
ว-06	P1	รูปแบบ error + รหัสธุรกิจ	ในสัปดาห์นี้
ว-07	P1	การแบ่งหน้าและการกรอง	ในสัปดาห์นี้
ว-08	P1	รูปแบบวันที่และเขตเวลา	ในสัปดาห์นี้
ว-09	P1	แปลงชื่อฟิลด์ + ห้ามส่ง Prisma model	ในสัปดาห์นี้
ว-10	P1	สถานะเป็นสตริงคงที่	ในสัปดาห์นี้
ว-11	P2	ขั้นตอนอัปโหลดรูป	เริ่มทำ feature รับ-คืน
ว-12	P2	ความถี่ polling จริง	หลังต่อหน้าแรกที่ poll
ว-13	P2	batching / timeout / retry	เมื่อเจอหน้าที่ช้า
ว-14	P2	ข้อความ error	หน้าจอที่มี error หลายกรณี
ว-15	P2	การ log และตามรอยปัญหา	ใกล้ส่งงาน
ว-16	P3	realtime / subscription	หลัง MVP
ว-17	P3	จำกัดอัตราการเรียก	หลัง MVP
ว-18	P3	เวอร์ชันของสัญญา	เมื่อมี client ที่อัปเดตไม่พร้อมกัน
ว-19	P3	เทสต์สัญญาอัตโนมัติ	เมื่อมีเวลา

เอกสารนี้เป็นวาระประชุม ไม่ใช่บันทึกการประชุม — เมื่อประชุมเสร็จ ให้บันทึกมติดลงแบบฟอร์มภาคผนวก ก แล้วเก็บไว้ดูกัน เหตุผลเบื้องหลังของทุกวาระ และที่มาของข้อเท็จจริงที่อ้างถึง อยู่ในเอกสารคู่กัน docs/trpc-guide.tex ภาคผนวก ง